

玩具级编译器的实现

格 llvm 致知：胡临天 王俊

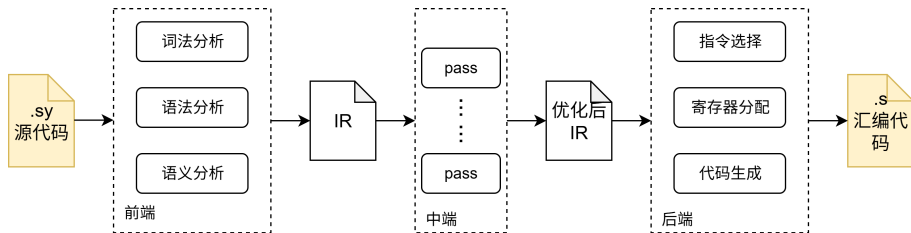
2025 年 9 月 9 日

https://github.com/hulintian/GWZZ_Compiler

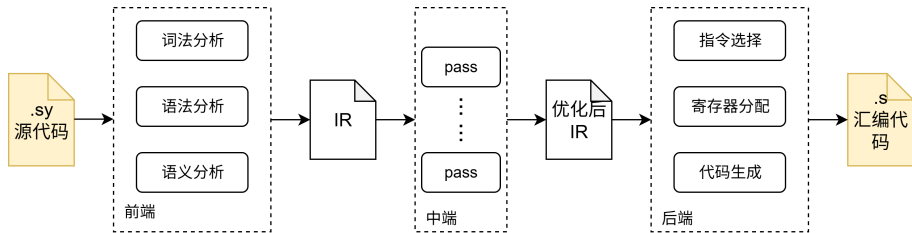
目录

- 1 概览
- 2 编译前端
- 3 编译中端
- 4 编译后端
- 5 编译优化

概览



概览



我们将编译器总体分成三个模块，分别为前端、中端和后端。前端词法分析和语法分析采用 ANTLR4 生成，通过遍历 ANTLR4 的解析树生成 AST。然后遍历 AST 进行语义分析、中间代码生成，生成中间表示代码（IR）。中间表示采用类 LLVM 的结构。后端遍历 IR，将 IR 指令翻译成等效的 RISC-V 指令。

为什么能够等效的翻译？

为什么能够等效的翻译？

哲学：语言的是用来描述怎么做的。

SysY 语言文法

SysY 语言的文法表示如下，其中 `CompUnit` 为开始符号：

编译单元	<code>CompUnit</code>	$\rightarrow$ <code>[CompUnit] (Decl FuncDef)</code>
声明	<code>Decl</code>	$\rightarrow$ <code>ConstDecl VarDecl</code>
常量声明	<code>ConstDecl</code>	$\rightarrow$ <code>'const' BType ConstDef { ';' ConstDef } ';'</code>
基本类型	<code>BType</code>	$\rightarrow$ <code>'int' 'float'</code>
常数定义	<code>ConstDef</code>	$\rightarrow$ <code>Ident { '[' ConstExp ']' } '=' ConstInitVal</code>
常量初值	<code>ConstInitVal</code>	$\rightarrow$ <code>ConstExp</code> $\quad$ <code> '{' [ConstInitVal { ';' ConstInitVal }] '}'</code>
变量声明	<code>VarDecl</code>	$\rightarrow$ <code>BType VarDef { ';' VarDef } ';'</code>
变量定义	<code>VarDef</code>	$\rightarrow$ <code>Ident { '[' ConstExp ']' }</code> $\quad$ <code> Ident { '[' ConstExp ']' } '=' InitVal</code>
变量初值	<code>InitVal</code>	$\rightarrow$ <code>Exp { '[' InitVal { ';' InitVal }] '}'</code>
函数定义	<code>FuncDef</code>	$\rightarrow$ <code>FuncType Ident '(' [FuncFParams] ')' Block</code>
函数类型	<code>FuncType</code>	$\rightarrow$ <code>'void' 'int' 'float'</code>
函数形参表	<code>FuncFParams</code>	$\rightarrow$ <code>FuncFParam { ';' FuncFParam }</code>
函数形参	<code>FuncFParam</code>	$\rightarrow$ <code>BType Ident '[' ']' { '[' Exp ']' }</code>
语句块	<code>Block</code>	$\rightarrow$ <code>'{' { BlockItem } '}'</code>
语句块项	<code>BlockItem</code>	$\rightarrow$ <code>Decl Stmt</code>
语句	<code>Stmt</code>	$\rightarrow$ <code>LVal '=' Exp ';' [Exp] ';' Block</code> $\quad$ <code> 'if' '(' Cond ')' Stmt { 'else' Stmt }</code> $\quad$ <code> 'while' '(' Cond ')' Stmt</code> $\quad$ <code> 'break' ';' 'continue' ';' 'return' [Exp] ';' </code>
表达式	<code>Exp</code>	$\rightarrow$ <code>AddExp</code> 注：SysY 表达式是 <code>int/float</code> 型

	表达式
条件表达式	<code>Cond</code> $\rightarrow$ <code>LOrExp</code>
左值表达式	<code>LVal</code> $\rightarrow$ <code>Ident { '[' Exp ']' }</code>
基本表达式	<code>PrimaryExp</code> $\rightarrow$ <code>'(' Exp ')' LVal Number</code>
数值	<code>Number</code> $\rightarrow$ <code>IntConst floatConst</code>
一元表达式	<code>UnaryExp</code> $\rightarrow$ <code>PrimaryExp Ident '(' [FuncRParams] ')' </code> $\quad$ <code>UnaryOp UnaryExp</code>
单目运算符	<code>UnaryOp</code> $\rightarrow$ <code>'+' '-' '!'</code> 注：'!'仅出现在条件表达式中
函数实参表	<code>FuncRParams</code> $\rightarrow$ <code>Exp { ';' Exp }</code>
乘除模表达式	<code>MulExp</code> $\rightarrow$ <code>UnaryExp MulExp ('*' '/' '%') UnaryExp</code>
加减表达式	<code>AddExp</code> $\rightarrow$ <code>MulExp AddExp ('+' '-') MulExp</code>
关系表达式	<code>RelExp</code> $\rightarrow$ <code>AddExp RelExp ('<' '>' '<=' '>=') AddExp</code>
相等性表达式	<code>EqExp</code> $\rightarrow$ <code>RelExp EqExp ('==' '!=') RelExp</code>
逻辑与表达式	<code>LAndExp</code> $\rightarrow$ <code>EqExp LAndExp '&&' EqExp</code>
逻辑或表达式	<code>LOrExp</code> $\rightarrow$ <code>LAndExp LOrExp ' ' LAndExp</code>
常量表达式	<code>ConstExp</code> $\rightarrow$ <code>AddExp</code> 注：使用的 <code>Ident</code> 必须是常量

词法分析和语法分析

用 ANTLR4 生成

词法规则：

- 关键字：上面文法中加粗的。
- 整数、浮点数：参考 iso9899。

语法规则：

- 根据文法的定义写。

生成抽象语法树：

- 为了便于后续的语义分析，去除文法中的冗余语义，定义一套简化的抽象语法树。用访问者模式遍历 ANTLR4 生成的语法解析树。

```
35 lexer grammar SysyLex;
36
37 // Lexer rules
38
39 // Types
40 INT : 'int';
41 FLOAT : 'float';
42 VOID : 'void';
43
44 // Keywords
45 IF : 'if';
46 ELSE : 'else';
47 WHILE : 'while';
48 BREAK : 'break';
49 CONTINUE : 'continue';
50 RETURN : 'return';
51 CONST : 'const';
52
53 // Symbols
54 Assign : '=';
55 Add : '+';
56 Sub : '-';
57 Mul : '*';
58 Div : '/';
59 Mod : '%';
60
61 Eq : '==';
62 Neq : '!=';
63 Lt : '<';
64 Gt : '>';
65 Leq : '<=';
66 Geq : '>=';
67
68 Not : '!';
69 And : '&&';
70 Or : '||';
```

词法分析和语法分析

用 ANTLR4 生成

词法规则：

- 关键字：上面文法中加粗的。
- 整数、浮点数：参考 iso9899。

语法规则：

- 根据文法的定义写。

生成抽象语法树：

- 为了便于后续的语义分析，去除文法中的冗余语义，定义一套简化的抽象语法树。用访问者模式遍历 ANTLR4 生成的语法解析树。

```
13 fragment Sign :[+];  
1 fragment Digit:[0-9];  
2 fragment Non_zero:[1-9];  
3 fragment Hex_Digit:[0-9A-Za-z];  
4 fragment Oct_Digit:[0-7];  
5 fragment Hex_prefix: '0x' | '0X';  
6  
7 IntConst  
8     : DecConst  
9     | OctConst  
10    | HexConst  
11    ;  
12  
13 DecConst  
14     : Non_zero Digit*  
15     ;  
16  
17 OctConst  
18     : '0'  
19     | '0' Oct_Digit+  
20     ;  
21  
22 HexConst  
23     : Hex_prefix Hex_Digit+  
24     ;  
25
```

词法分析和语法分析

用 ANTLR4 生成

语法规则：

- 关键字：上面文法中加粗的。
- 整数、浮点数：参考 iso9899。

语法规则：

- 根据文法的定义写。

生成抽象语法树：

- 为了便于后续的语义分析，去除文法中的冗余语义，定义一套简化的抽象语法树。用访问者模式遍历 ANTLR4 生成的语法解析树。

```
2 DecimalFloatingConst
1   : Frac_constant Exp?
87  | Digit+ Exp
1   ;
2
3 fragment Frac_constant
4   : Digit* '.' Digit+
5   | Digit+ '.'
6   ;
7
8 fragment Exp : [eE] Sign? Digit+;
9
10 HexFloatingConst
11  : Hex_prefix Hex_frac_constant B_Exp
12  | Hex_prefix Hex_Digit+ B_Exp
13  ;
14
15 fragment Hex_frac_constant
16  : Hex_Digit* '.' Hex_Digit+
17  | Hex_Digit+ '.'
18  ;
19
20 fragment B_Exp
21  : [Pp] Sign? Digit+
22  ;
23
```

词法分析和语法分析

用 ANTLR4 生成

词法规则：

- 关键字：上面文法中加粗的。
- 整数、浮点数：参考 iso9899。

语法规则：

- 根据文法的定义写。

生成抽象语法树：

- 为了便于后续的语义分析，去除文法中的冗余语义，定义一套简化的抽象语法树。用访问者模式遍历 ANTLR4 生成的语法解析树。

```
35 grammar Sysy22;
36
37 import SysyLex;
38
39 compUnits : compUnit* EOF;
40
41 compUnit : funcDef | decl;
42
43 decl
44     : constDecl
45     | varDecl
46     ;
47
48 constDecl
49     : CONST bType constDef ( ',' constDef)* ';'
50     ;
51
52 constDef
53     : Ident ( '[' exp ']' )* Assign initVal
54     ;
55
56 varDecl
57     : bType varDef ( ',' varDef)* ';'
58     ;
59
60 varDef
61     : Ident ( '[' exp ']' )* (Assign initVal)?
62     ;
63
64 initVal
65     : exp # init
66     | '{' (initVal ( ',' initVal)* )? '}' # initList
67     ;
68
69 bType
70     : INT # int
71     | FLOAT # float
72     ;
73
74 funcDef : funcType Ident ( '{' funcFParams? '}' ) block;
75
76 funcType
77     : bType # funcType_
78     | VOID # void
```

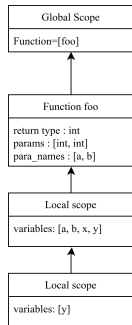
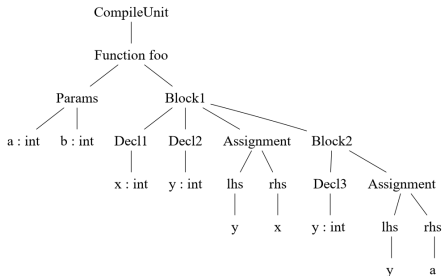
检查语义上是否正确

- 使用未声明的变量
- 使用未定义的函数
- 数组声明时大小是不是常量
- 赋值语句类型是否匹配

需要符号表的帮助

符号表

```
1 int foo(int a, int b) {  
2   ...int x;  
3   ...int y;  
4   ...y = a;  
5   ...{  
6     ...int y;  
7     ...y = x;  
8   ...}  
9 }
```



编译中端

模仿 LLVM



```
IR
├── BasicBlock.cpp
├── BasicBlock.hpp
├── Context.cpp
├── Context.hpp
├── Function.cpp
├── Function.hpp
├── GlobalValue.cpp
├── GlobalValue.hpp
├── IRBuilder.hpp
├── Instructions.cpp
├── Instructions.hpp
├── Module.cpp
├── Module.hpp
├── UndefValue.hpp
├── User.hpp
├── Value.hpp
├── instrs.yaml
├── opt.cpp
└── opt.hpp
```

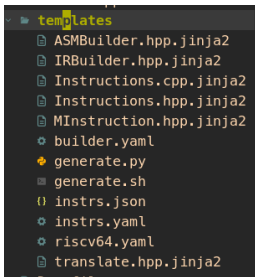
A screenshot of a code editor showing a file explorer for the 'IR' directory. The list includes files like BasicBlock.cpp, BasicBlock.hpp, Context.cpp, Context.hpp, Function.cpp, Function.hpp, GlobalValue.cpp, GlobalValue.hpp, IRBuilder.hpp, Instructions.cpp, Instructions.hpp, Module.cpp, Module.hpp, UndefValue.hpp, User.hpp, Value.hpp, instrs.yaml, opt.cpp, and opt.hpp.

模块、函数、基本块好定义，一个类就完事了。IR 指令有点多
add、sub、mul、div、alloca、load、store、gep, br.....。

中间代码指令

偷懒：把机械化的事情交给机器做。

用 python + jinja2 + yaml 生成一堆用于描述 IR 指令类和 IRBuilder 类（后端的 RISC-V 指令的定义也是一样）。



```
5 Instructions:
4 - Instruction: alloca
3 - Instruction: load
2 - Instruction: store
1 - Instruction: getelementptr
30 - Instruction: binary_op
1 - Instruction: add
2 - Instruction: sub
3 - Instruction: mul
4 - Instruction: udiv
5 - Instruction: sdiv
6 - Instruction: urem
7 - Instruction: srem
8 - Instruction: fadd
9 - Instruction: fsub
10 - Instruction: fmul
11 - Instruction: fdiv
12 - Instruction: frem
13 - Instruction: and
14 - Instruction: or
15 - Instruction: xor
16 # icmp
17 - Instruction: icmp
18 - Instruction: eq
19 - Instruction: ne
20 - Instruction: gt
21 - Instruction: lt
22 - Instruction: ge
23 - Instruction: le
24 # fcmp
25 - Instruction: fcmp
26 - Instruction: oeq
27 - Instruction: one
28 - Instruction: ogt
29 - Instruction: olt
30 - Instruction: oge
31 - Instruction: ole
32 # branch
33 - Instruction: br
34 - Instruction: cond_br
35 # function call
36 - Instruction: call
37 - Instruction: ret
38 # phi
39 - Instruction : phi
40 # convert types
41 - Instruction : cvt
```


中间代码指令

偷懒：把机械化的事情交给机器做。

用 python + jinja2 + yaml 生成一堆用于描述 IR 指令类和 IRBuilder 类（后端的 RISC-V 指令的定义也是一样）。

```
namespace IR {
class IRBuilder {
public:
    IRBuilder(Context* ctx) : _ctx(ctx) {}
    /* User Code Start: IRBuilder construct function */
    /* User Code End: IRBuilder construct function */

    /* User Code Start: code space 1 */
    // TODO: 在此处插入
    /* User Code End: code space 1 */

    {% for member in members %} {% if member.generate_get %}
    {{ member.type }} get({{ member.name }}) const {
        /* User Code Start: {{ classname }}::get({{ member.name|l }}) */
        return {{ member.name }};
        /* User Code End: {{ classname }}::get({{ member.name|l }}) */
    }
    {% endif %} {% endfor %}

    {% for member in members %} {% if member.generate_set %}
    void set({{ member.name }}) {{ member.type }} {{ member.name|l }} {
        /* User Code Start: set({{ member.name|l }}) */
        /* User Code End: set({{ member.name|l }}) */
    }
    {% endif %} {% endfor %}

    {% for Instr in Instructions %}
    Instruction* create({{ Instr.Instruction }}) {
        /* User Code Start: create({{ Instr.Instruction }}) args */
        /* User Code End: create({{ Instr.Instruction }}) args */
    }
    /* User Code Start: create({{ Instr.Instruction }}) */
    return nullptr;
    /* User Code End: create({{ Instr.Instruction }}) */
    {% endfor %}
private:

```

```
5 Instructions:
4 - Instruction: alloca
3 - Instruction: load
2 - Instruction: store
1 - Instruction: getelementptr
30 - Instruction: binary_op
1 - Instruction: add
2 - Instruction: sub
3 - Instruction: mul
4 - Instruction: udiv
5 - Instruction: sdiv
6 - Instruction: urem
7 - Instruction: srem
8 - Instruction: fadd
9 - Instruction: fsub
10 - Instruction: fmul
11 - Instruction: fdiv
12 - Instruction: frem
13 - Instruction: and
14 - Instruction: or
15 - Instruction: xor
16 # icmp
17 - Instruction: icmp
18 - Instruction: eq
19 - Instruction: ne
20 - Instruction: gt
21 - Instruction: lt
22 - Instruction: ge
23 - Instruction: le
24 # fcmp
25 - Instruction: fcmp
26 - Instruction: oeq
27 - Instruction: one
28 - Instruction: ogt
29 - Instruction: olt
30 - Instruction: oge
31 - Instruction: ole
32 # branch
33 - Instruction: br
34 - Instruction: cond_br
35 # function call
36 - Instruction: call
37 - Instruction: ret
38 # phi
39 - Instruction: phi
40 # convert types
41 - Instruction: cvt
```

中间代码指令

偷懒：把机械化的事情交给机器做。

用 python + jinja2 + yaml 生成一堆用于描述 IR 指令类和 IRBuilder 类（后端的 RISC-V 指令的定义也是一样）。

```
16 Instruction* create_cvt(
17     /* User Code Start: create_cvt args */
18     Value* v, Type* from, Type* to
19     /* User Code End: create_cvt args */
20 ) {
21     /* User Code Start: create_cvt */
22     assert(!from->is_array() && !to->is_array() && "Can't cvt the array type!");
23     if (from->base_type == to->base_type) {
24         return static_cast<Instruction*>(v);
25     }
26     auto name = "%t" + std::ito_string(v->this->get_cur_ctx()->get_tmp_var());
27     auto instr = new ConvertInst(v, from->base_type, to, name, this->get_cur_ctx());
28     b();
29     this->get_cur_ctx()->add_instr(instr);
30 }
31 return instr;
32 /* User Code End: create_cvt */
33 }
34
35 private:
36     Context* _cur_ctx;
37
38     /* User Code Start: 2 */
39     /* User Code End: 2 */
40 };
41 }
```

```
5 Instructions:
6 4 - Instruction: alloca
7 3 - Instruction: load
8 2 - Instruction: store
9 1 - Instruction: getelementptr
10 30 - Instruction: binary_op
11 1 - Instruction: add
12 2 - Instruction: sub
13 3 - Instruction: mul
14 4 - Instruction: udiv
15 5 - Instruction: sdiv
16 6 - Instruction: urem
17 7 - Instruction: srem
18 8 - Instruction: fadd
19 9 - Instruction: fsub
20 10 - Instruction: fmul
21 11 - Instruction: fdiv
22 12 - Instruction: frem
23 13 - Instruction: and
24 14 - Instruction: or
25 15 - Instruction: xor
26 16 # icmp
27 17 - Instruction: icmp
28 18 - Instruction: eq
29 19 - Instruction: ne
30 20 - Instruction: gt
31 21 - Instruction: lt
32 22 - Instruction: ge
33 23 - Instruction: le
34 24 # fcmp
35 25 - Instruction: fcmp
36 26 - Instruction: oeq
37 27 - Instruction: one
38 28 - Instruction: ogt
39 29 - Instruction: olt
40 30 - Instruction: oge
41 31 - Instruction: ole
42 32 # branch
43 33 - Instruction: br
44 34 - Instruction: cond_br
45 35 # function call
46 36 - Instruction: call
47 37 - Instruction: ret
48 38 # phi
49 39 - Instruction: phi
50 40 # convert types
51 41 - Instruction: cvt
```

中间代码生成

遍历抽象语法树，生成中间代码

- 函数：创建函数类，并创建一个入口基本块，先给函数形参分配空间（生成 `alloca` 语句）
- 运算：生成加、减、乘、除、取余等指令
- 声明：局部的用 `alloca` 分配内存，全局的记录到全局变量中
- 循环、分支：根据条件语句的结果生成跳转指令，为每个分支创建基本块，再翻译每个分支

指令选择

哲学：

- 我是谁：这条 IR 指令是干啥的？
- 我从哪里来：指令的数据从哪来？
- 我要到那去：指令的结果放到哪里去？

指令选择

操作数从哪来？

- 计算
 - 常量：立即数
 - 来自别的指令的结果：寄存器
- 取值地址来源包括：全局变量（通过 `la` 指令获取地址）、栈帧中的局部变量（`fp` 加偏移量）、数组元素（数组首地址 + 偏移量）。
- 存值地址来源与取值相同。
- 函数调用
 - 调用目标：被调函数
 - 参数来源：可能是计算结果或取值结果。
- 被调函数参数
 - 参数寄存器：`a0-a7` 或 `fa0-fa7`
 - 若寄存器不够：使用栈内存。超出的参数按约定由 `sp` 向高地址方向依次存放，每个槽占 8 字节（无论是 32 位整型还是 64 位指针，以 64 位对齐）。

指令选择

指令的结果放在哪里？

- 计算、取值指令结果放到寄存器中，并建立 IR Value（中间表示）到 Reg（寄存器）的映射。在后端翻译阶段，根据指令结果的使用位置，通过映射得到具体的寄存器。
- 存值将寄存器中的值写回内存。
- 跳转执行跳转操作。
- 函数调用参数存放位置：优先放在参数寄存器，不够时按约定放在栈上，即 $sp + 8 \times x$ （第 x 个溢出参数）。
- 被调函数每个参数视为函数的局部变量，根据值类型（数值或指针）和对齐规则分配地址（相对于 fp 的偏移量），并从寄存器或栈内存搬运到被调函数的栈帧中。

寄存器分配

线扫描寄存器分配。

简单，扫描两趟。一趟扫描找（虚拟）寄存器的活跃区间，一趟扫描分配寄存器。

汇编代码生成

以上的过程中所谓的指令、基本块、函数、模块都是一个在内存中的数据结构，汇编代码生成就是把这些信息按照汇编代码的样子组织成文本的形式。

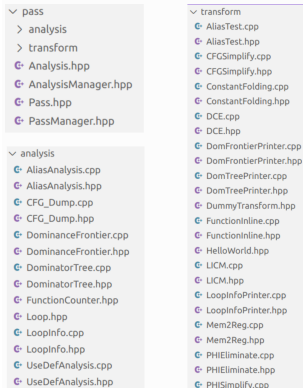
优化框架

核心类：

- Pass;
- PassManager;
- Analysis;
- AnalysisManager;

Pass 分为变换 Pass 和分析 Pass，变换 Pass 从 PassManager 中获取 AnalysisManager 并调用指定分析 Pass 分析 IR，获得分析结果。根据分析结果执行相应的优化策略。

AnalysisManager 对分析结果统一进行管理，在初次调用分析 Pass，会将其返回的分析结果进行缓存，以便后续 Pass 获取，节省了性能；若 IR 进行了修改，变换 Pass 的 run 函数返回 true 时，或者在 pass 中根据需要手动作废分析结果时，AnalysisManager 会清空对应缓存。



实现的 Pass

分析 Pass

- 支配树分析
- 支配边界分析
- 循环分析
- 别名分析
- Use-Def 分析

变换 Pass

- CFG 简化
- Mem2Reg
- 常量传播
- LICM
- PHI 简化
- DCE
- PHI 节点消除

其他功能: CFG_dump、 CallGraph_dump

